

Requirement 👍

Modules or Microservices

- User service (register, login)
- Music service (upload, store, fetch metadata (songs, albums, playlists).
- Search service (indexing & searching songs/artists.)
- Streaming services (audio streaming & buffering.)
- Recommendation service (optional)
- Payment service (Optional)

Tools and Technologies

Backend

- java, springboot

Databases

- **MySql** → caching (popular songs, recommendations). structure data (users, playlists)
- **MongoDB** → unstructured & search queries (song metadata, lyrics)
- **Redis** → caching (popular songs, recommendations).

FrontEnd

- **React js**
- **TailwindCSS styling**

Music Streaming & Storage

- Audio storage:
 - AWS S3 / Google Cloud Storage / Azure Blob → store songs.
- CDN (Content Delivery Network):
 - Cloudflare / AWS CloudFront → deliver songs faster.
- Streaming:
 - Encode songs in MP3/AAC/OGG.
 - Use HLS (HTTP Live Streaming) or DASH for adaptive streaming.

Recommendation System (AI/ML)

- Collect user behavior: listens, skips, likes.
- Implement ML models:
 - Collaborative filtering (like users, like songs).
 - Content-based filtering (similar songs by genre/artist).
- Tools:
- Python + Scikit-learn / TensorFlow.
- Or integrate with Spotify Web API for testing.

DevOps & Deployment

- Containerization: Docker for each service.
- Orchestration: Kubernetes (K8s) for scaling.
- CI/CD: GitHub Actions / Jenkins.
- Monitoring: Prometheus + Grafana (we can use spring admin server).

- Tracing: Zipkin (you already used Zipkin 👍).

Other Must-Haves

- Payment Integration → Stripe/PayPal/Razorpay (for premium subscription).
- Logging & Error Handling → ELK Stack (Elasticsearch, Logstash, Kibana).
- API Documentation → Swagger/OpenAPI.

Architecture for Music App



